UNIVERSITÀ DI PISA

Computer Engineering

Cloud Computing

Letter frequency counter

Academic Year: 2023/2024

Alessandro Ascani

Gabriele Pianigiani

Saverio Mosti

Summary

Introduction **2**
The algorithm and the strategies **2**
Separate combiner implementation 2
In-Mapper Combining implementation 3
Testing **4**
Results **5**
Performance indexes 5
Input Splits 5
Distributed approach (Hadoop) 6
In-Mapper combining 6
Combiner 7
Not distributed approach 8
Python 8
C++ 9
Italian letter frequency analysis **10**
Conclusions **13**

Introduction

The scope of this project is to create a system that counts the frequency of the 26 letters of the English alphabet in a `.txt` file.

By leveraging different strategies and approaches, we aim to compare their performance in terms of speed, scalability, and efficiency. The primary objective is to determine the most effective method for processing large text files, focusing on the benefits and limitations of each approach.

The algorithm and the strategies

The algorithm is very simple:

1. Extract the content from the `.txt` file.
2. An associative vector is initialized with a row for each of the 26 characters of the english alphabet.
3. Each character of the text is scrolled and when a letter is found the corresponding entry is incremented.

In the project requirements it was stated that we have to implement two versions of the system: one with an In-Mapper combining and the other with a combiner.

Separate combiner implementation

One of the implementation requirements was to implement a separate combiner.

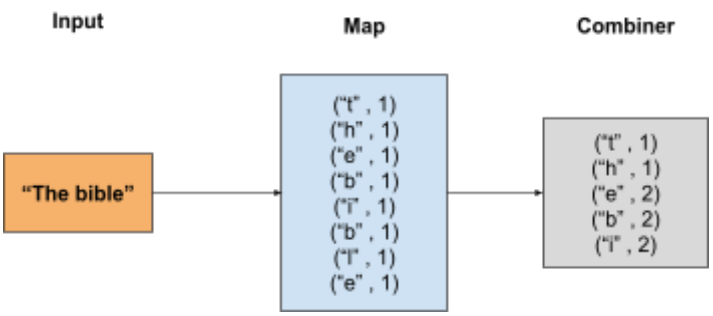

Figure 1 - Example of a separate combiner

The combiner does a pre-work for the reducer and it helps to reduce the amount of data transmitted to the reducers.

In-Mapper Combining implementation

One of the implementation requirements was to implement an In-Mapper combining. In this case we adopt a **Stateful** In-Mapper combining:

Every mapper has its own state where it puts the result that it produces. When the mapper find a letter, it do a little check:

- **If the letter is already in the state**: it means that the letter has been seen before, so it increments the associated value.
- **If the letter is NOT already in the state**: it means that the letter has not been seen before, so it inserts a new entry in the state.

```
class Mapper
    method Initialize
        H ← new AssociativeArray
    method Map(docid a, doc d)
        for all term t ∈ doc d do
            H{t} ← H{t} + 1
    method Close
        for all term t ∈ H do
            Emit(term t, count H{t})
```

Figure 2 - Stateful In-Mapper combining

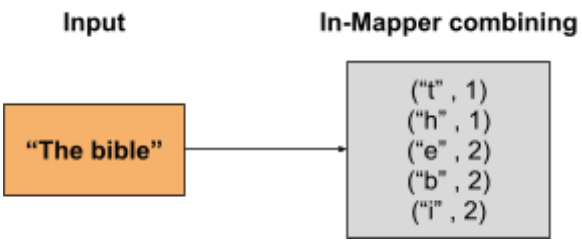

Figure 3 - Example of a stateful In-Mapper combining

Testing

The testing dataset is composed by 5 text files, of various dimensions:

- `pg.txt`: File composed of **2 KiloBytes** of data.
- `pg2.txt`: File composed of **2 MegaBytes** of data.
- `pg22.txt`: File composed of **22 MegaBytes** of data.
- `pg200.txt`: File composed of **200 MegaBytes** of data.
- `pg1000.txt`: File composed of **1 GigaBytes** of data.
- `pg5000.txt`: File composed of **5 GigaBytes** of data.

All those sample texts have been derived (*to generate a bigger file we simply repeated the original text*) from the same text "*The King James Bible - 1611*".

These samples have been used to test the performance of the system, and not to analyze the frequency of the letters (*that is shown later in this paper*).

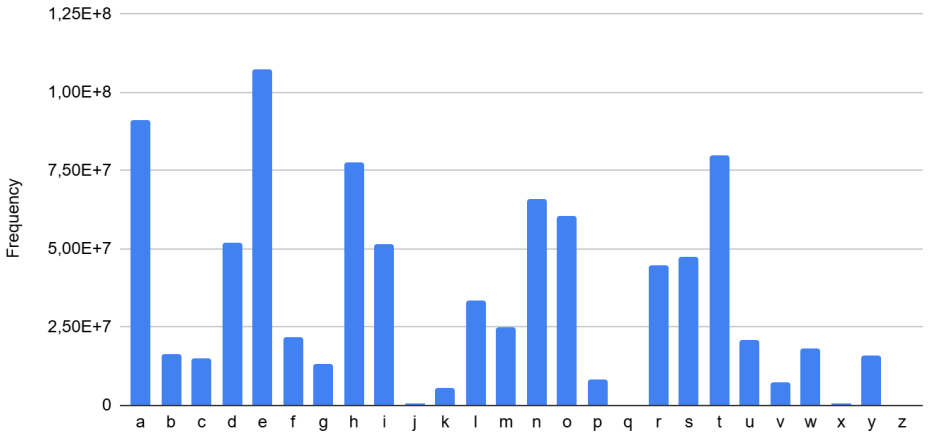

Figure 4 - Letter frequency of the King James Bible.

Results

Performance indexes

The parameters are:

- **Implementation:** Between In-Mapper combining and combiner.
- **Number of reducers:** The number of used reducers.

Every attempt consists in complete each job (*one job for each testing file*); and the performance indexes are:

- **Bytes written:** How many bytes have been written during the execution of the jobs.
- **Total execution time:** The required CPU time to complete all the jobs.
- **Total mapper time:** For how long the mappers have worked.
- **Total reducer time:** For how long the reducers have worked.

In particular, the **total execution time** refers to the time spent by the CPU to execute the application, it doesn't include the time in which the CPU is idle.

On the other hand, the **total mapper time** takes into account the CPU utilization time but also the waiting time to manage the memory, the synchronization time between activities and waiting time for the I/O.

In other words it represents the total time in which every map activity has been running in the slots of resources of the custer. The **total reducer time** has the same meaning as the previous one but for the reducers.

In the following table there is the number of input splits in which the initial input has been divided by MapReduce. We have reported the number of the input splits for each file, the values are the same for the in-mapper combining version and mapper version.

Input Splits

File	File Size (Bytes)	Number of Input Splits
`pg.txt`	2820	25
`pg2.txt`	2463840	21998
`pg22.txt`	22107852	195644
`pg200.txt`	221082616	969660
`pg1000.txt`	1114194168	1076516
`pg5000.txt`	5368868864	1167145

Distributed approach (*Hadoop*)

In-Mapper combining

Num. of Reducers	Bytes Written	Total Time (*seconds*)	Mapper time (seconds)	Reducer time (*seconds*)
1	1122	1004,47	6213,114	375,75
2	1122	1007,75	6093,164	618,7
3	1122	1011,51	5630,172	993,278

Follows the performance for each file.

File	Num. of Reducers	Bytes Written	Total Time (*seconds*)	Mapper time (*seconds*)	Reducer time (*seconds*)
`pg.txt`	1	137	1,34	5,454	5,668
`pg.txt`	2	137	1,75	5,072	12,784
`pg.txt`	3	137	2,27	5,178	18,406
`pg2.txt`	1	213	2,29	6,172	6,142
`pg2.txt`	2	213	2,49	7,04	10,062
`pg2.txt`	3	213	3,31	6,546	20,174
`pg22.txt`	1	235	4,82	12,058	5,78
`pg22.txt`	2	235	5,66	11,982	10,984
`pg22.txt`	3	235	6,15	11,836	18,698
`pg200.txt`	1	261	32,03	94,214	13,524
`pg200.txt`	2	261	33,38	104,372	30,708
`pg200.txt`	3	261	34,17	93,588	31,532
`pg1000.txt`	1	276	167,49	939,082	93,33
`pg1000.txt`	2	276	165,92	1085,106	131,21
`pg1000.txt`	3	276	168,53	879,232	182,63
`pg5000.txt`	1	296	796,5	5156,134	251,306
`pg5000.txt`	2	296	798,55	4879,592	422,952
`pg5000.txt`	3	296	797,08	4633,792	721,838

Combiner

Num. of Reducers	Bytes Written	Total Time (seconds)	Mapper time (seconds)	Reducer time (seconds)
1	1122	4401,23	25060,53	1271,394
2	1122	4448,86	24003,538	2379,664
3	1122	4430,05	22879,826	3763,654

Follows the performance for each file.

File	Num. of Reducers	Bytes Written	Total Time (seconds)	Mapper time (seconds)	Reducer time (seconds)
`pg.txt`	1	137	1,26	5,344	5,04
`pg.txt`	2	137	1,91	5,396	10,704
`pg.txt`	3	137	2,53	5,61	18,66
`pg2.txt`	1	213	3,75	9,19	5,604
`pg2.txt`	2	213	4,58	9,692	10,934
`pg2.txt`	3	213	5,05	8,91	16,3
`pg22.txt`	1	235	21,38	36,274	5,804
`pg22.txt`	2	235	21,78	36,01	11,802
`pg22.txt`	3	235	20,69	34,064	20,088
`pg200.txt`	1	261	153,14	379,776	59,814
`pg200.txt`	2	261	157,79	381,626	114,768
`pg200.txt`	3	261	185,65	378,244	210,428
`pg1000.txt`	1	276	725,63	4703,54	303,118
`pg1000.txt`	2	276	714,78	4581,688	582,884
`pg1000.txt`	3	276	708,48	4064,682	712,418
`pg5000.txt`	1	296	3496,07	19926,406	892,014
`pg5000.txt`	2	296	3548,02	18989,126	1648,572
`pg5000.txt`	3	296	3507,65	18388,316	2785,76

Not distributed approach

We also create two separate versions of the same system, but using a not distributed approach. For this reason we wrote the same program with Python (*high level and interpreted*) and C++ (*low level and compiled*), we submitted the same input files and we estimated the total execution time and the number of written bytes. Obviously, those two implementations do not follow the MapReduce philosophy and they do use more than one thread.

Python

Number of Threads	Bytes Written	Total Time (seconds)
1	1418	1885,3522
3	1418	2021,1138
5	1418	2351,2679

File	Num. of Threads	Bytes Written	Total Time (seconds)
`pg.txt`	1	137	0,0156
`pg.txt`	3	137	0,0155
`pg.txt`	5	137	0,0021
`pg2.txt`	1	213	0,6288
`pg2.txt`	3	213	0,6603
`pg2.txt`	5	213	0,6174
`pg22.txt`	1	235	5,2051
`pg22.txt`	3	235	5,6799
`pg22.txt`	5	235	5,5769
`pg200.txt`	1	261	53,5128
`pg200.txt`	3	261	57,3671
`pg200.txt`	5	261	57,8968
`pg1000.txt`	1	276	303,5420
`pg1000.txt`	3	276	309,0610
`pg1000.txt`	5	276	315,8120
`pg5000.txt`	1	296	1522,4479
`pg5000.txt`	3	296	1648,330

pg5000.txt	5	296	1971,3627

C++

Number of Threads	Bytes Written	Total Time (seconds)
1	1578	550,162
3	1578	351,081
5	1578	349,301

File	Num. of Threads	Bytes Written	Total Time (seconds)
pg.txt	1	167	0,000775967
pg.txt	3	167	0,00231203
pg.txt	5	167	0,00230778
pg2.txt	1	239	0,241856
pg2.txt	3	239	0,13569
pg2.txt	5	239	0,132097
pg22.txt	1	261	2,16766
pg22.txt	3	261	1,13708
pg22.txt	5	261	1,16138
pg200.txt	1	287	20,3617
pg200.txt	3	287	11,1956
pg200.txt	5	287	11,3203
pg1000.txt	1	302	108,698
pg1000.txt	3	302	68,0756
pg1000.txt	5	302	70,9262
pg5000.txt	1	322	418,692
pg5000.txt	3	322	270,535
pg5000.txt	5	322	265.759

Italian letter frequency analysis

Now follows the analysis of 8 Italian texts, selected from different years between 19 B.C. and 2023 A.D.

The objective of the analysis is to show how the frequency of letters has changed in the language used in the Italian peninsula over the last 2000 years.

The chosen texts for the experiment are:

- **Eneide**: Written in Latin between 29 B.C. and 19 B.C.
- **Divina Commedia**: Written in the Tuscan dialect and published in 1321.
- **Il Principe**: The Italian version written in 1532.
- **I Promessi Sposi**: Published in 1827.
- **Il Piacere**: Published in 1889.
- **The Italian Constitution**: Published in 1948.
- **The Bible of Jerusalem**: 1973 version.
- **FIAT statute**: Written in 2023.

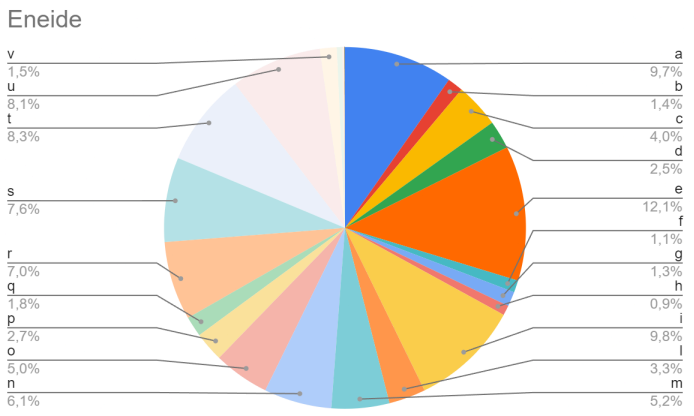

Figure 5 - Letter frequency in “Eneide”.

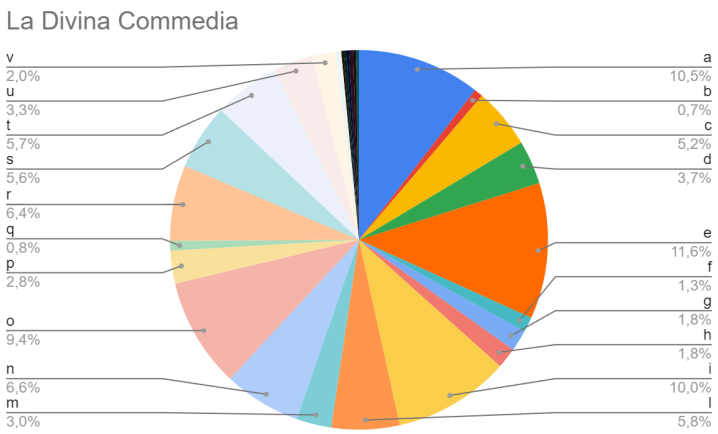

Figure 6 - Letter frequency in “La Divina Commedia”.

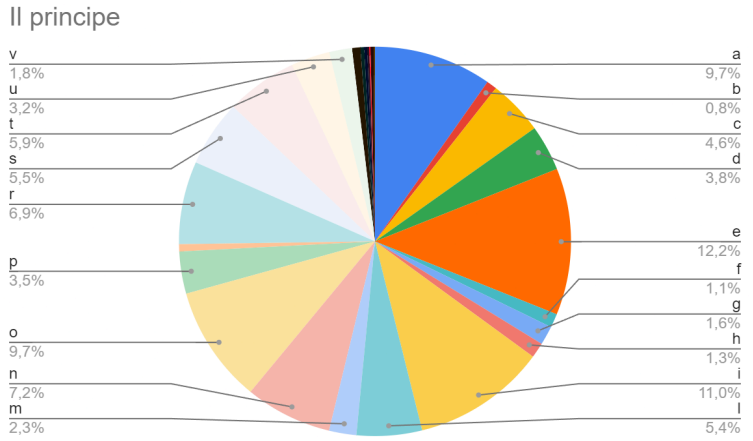

Figure 7 - Letter frequency in “Il Principe”

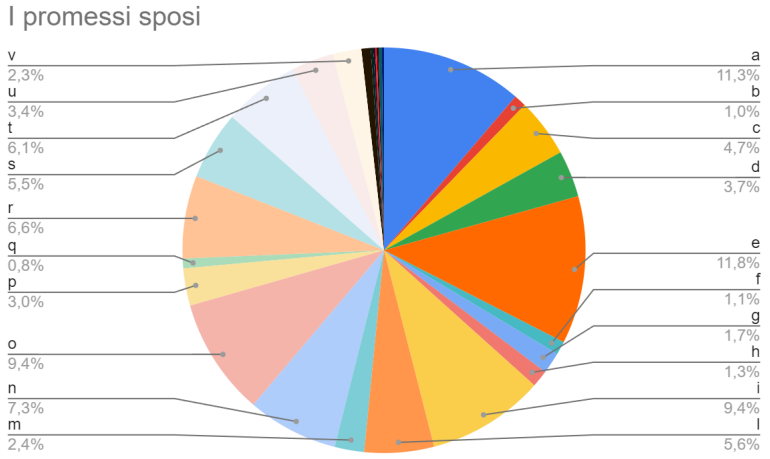

Figure 8 - Letter frequency in the “I promessi sposi”

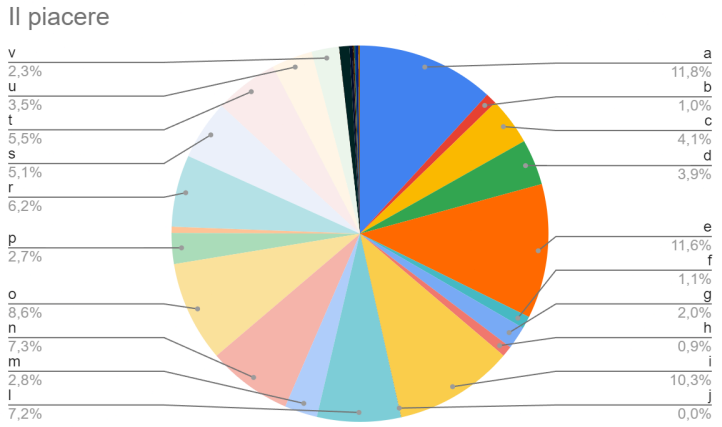

Figure 9 - Letter frequency in the “Il piacere”

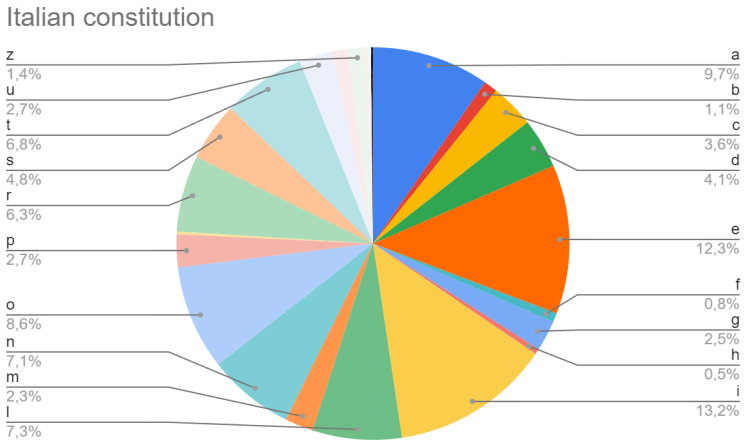

Figure 10 - Letter frequency in the “Bible of Jerusalem”

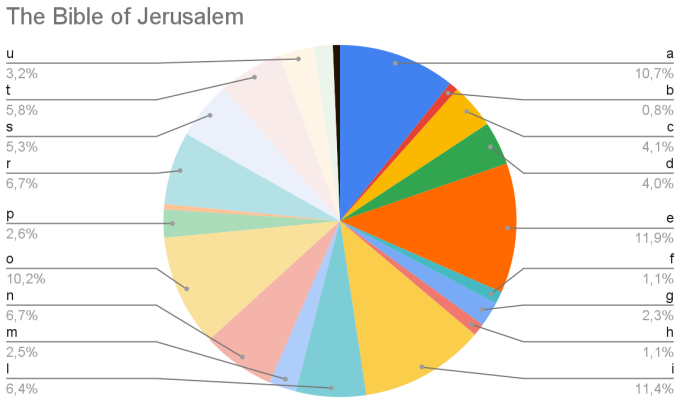

Figure 11 - Letter frequency in the “Bible of Jerusalem”

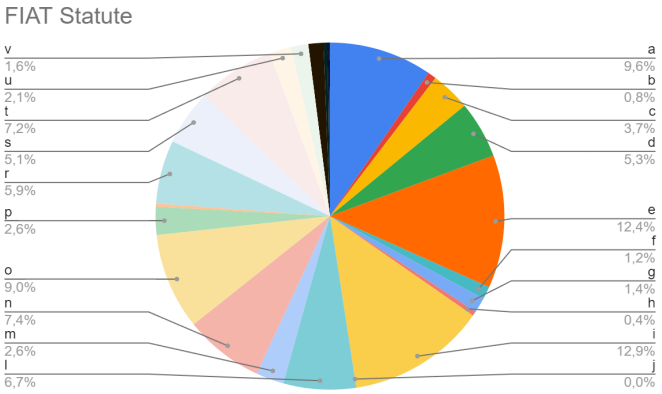

Figure 12 - Letter frequency in the “FIAT Status”

Conclusions

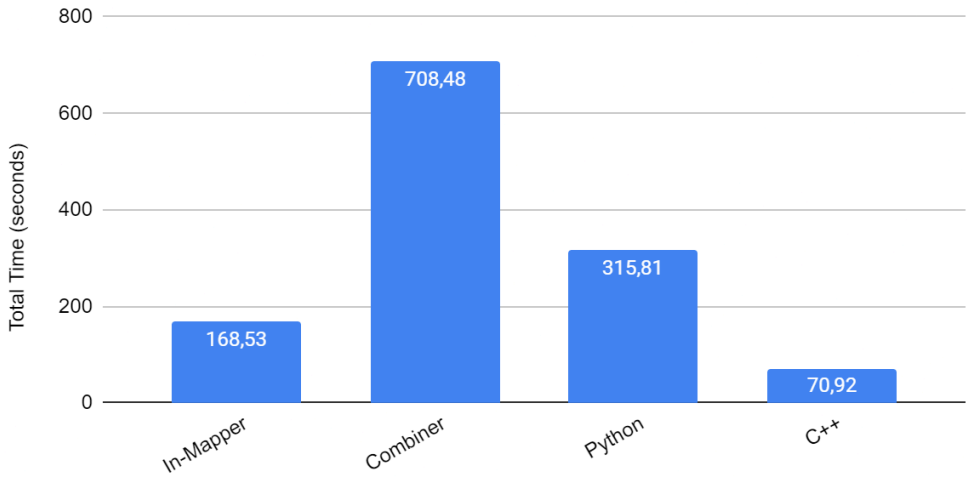

Figure 13 - How the execution time (*in the case of 1GB of data*) changes with the strategy.

The results of this Hadoop project highlight several key observations about the different tested strategies. Firstly, the In-Memory combining strategy significantly outperforms the combiner strategy in terms of execution time.

For both the combiner and In-Mapper combining strategies, increasing the number of reducers from 1 to 3 did not yield any notable improvements in performance. This suggests that the bottleneck in these strategies may not be in the reduction phase or that other factors are limiting the benefits of additional reducers.

A particularly interesting comparison is between the C++ strategy and the In-Mapper combining strategy. The In-Mapper combining strategy demonstrates superior scalability. While the execution time for the C++ strategy increases at an exponential rate as the input size grows, the In-Mapper combining strategy scales more efficiently. This indicates that the In-Mapper combining strategy is better suited for handling larger datasets, making it a more robust choice for scalable data processing.